UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Practical 3: Operator overloading

Release Date: 31-08-2026 at 06:00

Due Date: 04-09-2026 at 23:59

Late Deadline: 05-09-2026 at 00:59

Total Marks: 160

Read the entire specification before starting with the practical.

Contents

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- **This assignment should be completed individually.**
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

Operator overloading allows objects to define custom behaviour for predefined operators. In this practical, you will apply operator overloading to the ‘Worker‘ and ‘Factory‘ classes, implementing operators for tasks such as assignment, comparison, object modification, worker management, and input/output. This will allow the ‘Worker‘ and ‘Factory‘ objects to interact with operators in a way that is meaningful to their roles within the factory.

2.1 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
Worker operators	26
Factory creation	44
Factory operators	69
Testing	21
Total	160

3 Your Task:

You are required to implement all of the functions laid out in this section by firstly creating your header files and then the implementation files.

3.1 Design

Before you start coding, design an UML class diagram showing each function and variable in the relevant class and the relationship between worker and Factory. Include multiplicity in this relationship. Upload this as a draw.io project onto the CS portal.

3.2 Worker

This class represents the workers in the factory.

3.2.1 Members

3.2.1.1 - idCounter: int

- This is a static global counter to keep track of worker ID's.
- Should be initialized to 1.

3.2.1.2 - ID: int

- This stores the ID of a specific worker.

3.2.1.3 - name: string

- This string represents a workers full name e.g. "John Doe".

3.2.1.4 salary: double

- This is a decimal value to represent a workers salary in rands.

3.2.2 Functions

3.2.2.1 + Worker(name: string, salary: double)

- This is the constructor for the worker and should set member variables accordingly.
- Set the ID to the value of the idCounter and increment the static variable.

3.2.2.2 + ~ Worker()

- The destructor should free all dynamic memory used in the worker class.

3.2.2.3 + operator=(name: string): Worker&

- This operator sets the workers new name.
- If the passed in name is empty do not modify the current object.

3.2.2.4 + operator=(salary: double): Worker&

- This operator sets the workers new salary.
- If the passed in salary is less than 0 do not modify the current object.

3.2.2.5 + operator+=(increase: int): Worker&

- This operator should increase the workers salary by the passed in amount.
- If the amount makes the salary less than 0 do not modify the current object.

3.2.2.6 + operator-=(decrease: int): Worker&

- The workers salary should be decreased by the passed in amount.
- If the new salary is less than 0 do not modify the current object.

3.2.2.7 + operator==(other: const Worker&): bool

- Two workers are equal if their names and salaries are equal.
- This is a constant function and should not modify the object.

3.2.2.8 + operator!=(other: const Worker&): bool

- Two workers are not equal if their names or their salaries are different.
- This is a constant function and should not modify the object.

3.2.2.9 Comparison operators:

- For the operators below, compare the salaries to determine the results.
- These are constant functions and should not modify the object.
- + operator>(other: const Worker&): bool
- + operator<(other: const Worker&): bool
- + operator>=(other: const Worker&): bool
- + operator<=(other: const Worker&): bool

3.2.2.10 + operator=(other: const Worker&): Worker&

- This is the assignment operator and should only copy over the workers name and salary.

3.2.2.11 + operator()(): double

- This should return the workers salary.
- This is a constant function and should not modify the object.

3.2.2.12 + operator«(os: ostream&, worker: const Worker&): ostream&

- This operator returns the worker in the following format:
  `Worker:␣<ID>,<Name>,R<Salary>`
- The values inside the brackets should be replaced with the variables and ensure that you include a space after the colon. **Round the salary to 2 decimal places!**

3.3 Factory

3.3.1 Members

3.3.1.1 - workers: Worker**

- This stores a 1D array of dynamic worker objects.
- It should be deallocated in the destructor.

3.3.1.2 - numWorkers: int

- This keeps track of how many workers are in the array.

3.3.2 Functions

3.3.2.1 + Factory()

- This is the default constructor and should set the number of workers to 0.
- It should create an empty array of size 0 for workers.

3.3.2.2 + ~ Factory()

- This is the destructor. All dynamic memory should be freed.

3.3.2.3 + Factory& operator=(other: const Factory&)

- This works like a normal assignment operator, the ID's of the workers should be the same, and do not increase the static counter.

3.3.2.4 + Factory& operator|(string filename)

- This is the piping operator and should load the factory based on the passed in text file.
- Each line in the textfile is a worker that should be added to the array.
- Ignore the ID's from the text file and add workers normally.
- The text file follows the output produced by `operator«` from factory.
- If the textfile does not exist do not modify the current object.

3.3.2.5 + int operator[](worker: const Worker&)

- This is the subscript operator and should return the index of the worker which is equal to the passed in worker.
- If no worker exists return -1.
- This is a constant function and should not modify the object.

3.3.2.6 + Worker* operator[](index: int)

- This is an alternative subscript operator and it should return the worker at the given index.
- Return null if the index is out of bounds.
- This is a constant function and should not modify the object.

3.3.2.7 + void operator*=(other: const Factory&)

- This operator modifies the current factory by appending all of the workers from the other factory to the current factory using deep copies of each worker.

3.3.2.8 + Factory* operator*(other: const Factory&)

- This does the same as operator*= however you should make a new factory, first add all workers from the current factory to the new factory and then all the workers from the other factory using deep copies.
- This is a constant function and should not modify the object.

3.3.2.9 + Factory& operator+=(worker: Worker*)

- This operator should create a deep copy of the passed in worker pointer when adding it to the array.
- The ID should be the next value, and not the same as the passed in worker.
- The input can be NULL and in that case do nothing.
- Do this by resizing the array by 1 and then adding the worker to the end.

- Do now allow duplicate workers to be added to the factory.

3.3.2.10 + Factory& operator+=(worker: Worker&)

- This operator should directly add the worker to the worker array using a shallow copy.
- Do now allow duplicate workers to be added to the factory.

3.3.2.11 + Factory& operator-=(worker: Worker*)

- This operator deletes the given worker from the factory.
- Remove the worker at the given index and then resize the array accordingly.

3.3.2.12 + operator==(other: const Factory&): bool

- Two factories are equal if their total salaries are equal.
- This is a constant function and should not modify the object.

3.3.2.13 + operator!=(other: const Factory&): bool

- Two factories are not equal if their total salaries are different.
- This is a constant function and should not modify the object.

3.3.2.14 Comparison operators:

- For the operators below, if the total salary of both factories are equal return a new default factory in all cases. Otherwise return the factory which would make the condition true based on the factory's total salary.
- These are constant functions and should not modify the object.
- + operator>(other: const Factory&): const Factory&
- + operator<(other: const Factory&): const Factory&
- + operator>=(other: const Factory&): const Factory&
- + operator<=(other: const Factory&): const Factory&

3.3.2.15 + operator«(os: ostream&, factory: const Factory&): ostream&

- If the factory has no workers add `"Empty␣factory"` to the `os` object.
- Otherwise add each worker from the factory ending with `"\n"` to the `os` object.

3.3.2.16 + operator»(is: istream&, factory: Factory&): istream&

- This operator is used to populate the factory from an istream (assume the `istream` follows the same format as the text file).

4 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

5 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the main.cpp file to aid your testing. In order to determine the coverage of your testing the gcov [1] tool, specifically the following version *gcov (Debian 8.3.0-6) 8.3.0*, will be used. The following set of commands will be used to run gcov:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%
40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing

[1] For more information on gcov please see `https://gcc.gnu.org/onlinedocs/gcc/Gcov.html`

mark. Remember that your main will be testing the Instructor Provided code and as such, it can only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

6 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **c++98**.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
  - fstream
  - sstream
  - iomanip
  - iostream
  - string
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

7 Upload Checklist

The following c++ files should be in a zip archive named uXXXXXXXX.zip where XXXXXXXX is your student number:

- `Worker.cpp`
- `Factory.cpp`
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

8 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (`https://ff.cs.up.ac.za/`). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

and run with the following command:

```
./main
```

You have 10 submissions, and your best mark will be your final mark. Upload your archive to the Practical 3 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**